

RootRecord Software Solutions

<https://github.com/rootrecordsoftwaresolutions> | <https://github.com/rootrecordsoftwaresolutions/Solar-Pacific-RootRecord-Server> | <https://github.com/rootrecordsoftwaresolutions/Solar-Pacific-RootRecord-Server/tree/main/0-master-prompt> | <https://github.com/rootrecordsoftwaresolutions/Solar-Pacific-RootRecord-Server/blob/main/0-master-prompt/MASTER-PROMPT.md> | <https://github.com/rootrecordsoftwaresolutions/Solar-Pacific-RootRecord-Server/blob/main/0-master-prompt/MULTI-AGENT-IDENTITY-AND-AI-ORCHESTRATION.md> | <https://github.com/rootrecordsoftwaresolutions/RootRecord-Master-Prompt> | <https://github.com/rootrecordsoftwaresolutions/RootRecord-Weather-Database> | <https://github.com/rootrecordsoftwaresolutions/RootRecord-Website> | <https://github.com/rootrecordsoftwaresolutions/US-Mainland-Server>

SUMMARY

This document defines the next evolutionary restructuring of the RootRecord ecosystem following the clean-system reset. The objective is to determine the cleanest, most efficient, maintainable, recoverable, scalable, and logically separated architecture possible for RootRecord going forward.

PROJECTS

Solar-Pacific-RootRecord-Server | Primary local/server-side environment in the RootRecord ecosystem rebuild.

US-Mainland-Server | Distributed automation and offloading environment for recurring automation, AWS/EC2 workloads, synchronization, and recovery-oriented services.

RootRecord-Website | Canonical frontend repository and Vercel deployment layer for all RootRecord pages targeting rootrecord.cloud.

RootRecord-Weather-Database | Canonical data project that must operate independently of Ollama.

RootRecord-Master-Prompt | Cross-project orientation and operating contract reference.

TECHNICAL SKILLS

operational efficiency

human efficiency

development efficiency

agent efficiency

AI efficiency

storage efficiency

infrastructure efficiency

deployment efficiency

recovery efficiency

maintenance efficiency

organizational clarity

long-term scalability

architecture analysis

implementation suggestions

documentation

code

validation

DOCUMENT STATUS

Collaborative Architectural Draft

PRIMARY PROPOSER

GPT (ChatGPT)

CONTRIBUTORS

GPT, Claude, Grok, GitHub Copilot

FINAL EDITOR

Claude

CANONICAL OWNERSHIP

RootRecord Software Solutions

CORE PRINCIPLE

RootRecord owns the systems. RootRecord owns the canonical repositories and data. Operations execute the systems. Agents operate the systems. Skills teach agents how to operate the systems. Models provide intelligence to the agents. AI providers contribute intelligence and development capability without owning RootRecord.

OPEN ARCHITECTURAL QUESTIONS FOR COLLABORATIVE REVIEW

1. What should the final physical RootRecord directory structure be? 2. Which components belong in the central operational root versus individual repositories? 3. Which operations should remain local to Solar/Pacific? 4. Which operations should be offloaded to the US Mainland Server? 5. What should remain in `ollama/skills`? 6. Should agent skills eventually become their own organizational repository? 7. How should shared agent capabilities be versioned? 8. How should agent-specific skills differ from system-wide operational tooling? 9. What is the cleanest model/provider abstraction? 10. How should multiple AI providers collaborate without creating duplicated state? 11. What should be canonical locally versus canonical in GitHub? 12. How should RootRecord Website consume the various canonical data systems? 13. What portions of the global layer should be rebuilt versus recovered? 14. What should the long-term deployment topology look like? 15. What should be automated versus explicitly human-controlled? 16. How should disaster recovery work if Solar, Mainland, GitHub, or an AI provider becomes unavailable? 17. Which existing systems are genuinely necessary versus historical accumulation? 18. How can the final architecture minimize operational overhead while retaining capability?